

BÁO CÁO THỰC HÀNH

Bài 13: Lập trình hợp ngữ trên ESP32-C3 và mô phỏng với Wokwi

Assignment 1 - Xuất mức HIGH trên các chân GPIO2, GPIO3 và GPIO4

1. Mã nguồn đã biên tập chú thích

```
.global init

# Khai báo địa chỉ các thanh ghi cần sử dụng
.equ GPIO_ENABLE_REG, 0x60004020    # Cho phép chân GPIO hoạt động ở chế độ xuất tín hiệu
.equ GPIO_OUT_W1TS_REG, 0x60004008    # Ghi 1 để đặt mức logic cao cho GPIO
.equ IO_MUX_GPIO4_REG, 0x60004014    # Chọn lại chức năng cho chân GPIO4

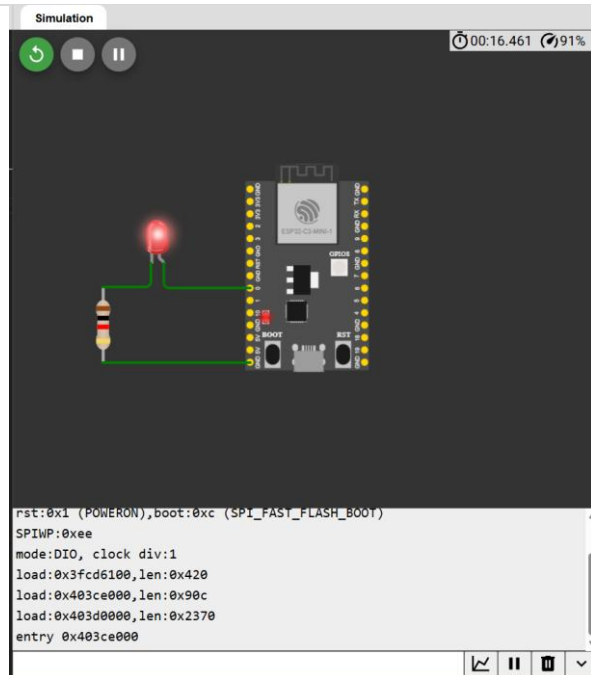
.text
init:
    # Bật chế độ Output cho GPIO2, GPIO3 và GPIO4
    # 0x1C = 11100b, tương ứng với các bit 2, 3, 4
    li a1, GPIO_ENABLE_REG
    li a2, 0x1C
    sw a2, 0(a1)

    # Đưa GPIO4 về chức năng GPIO thường thay vì chức năng SPI mặc định
    # Bit cấu hình MCU_SEL được đặt bằng giá trị 1 thông qua 0x1000
    li a1, IO_MUX_GPIO4_REG
    li a2, 0x1000
    sw a2, 0(a1)

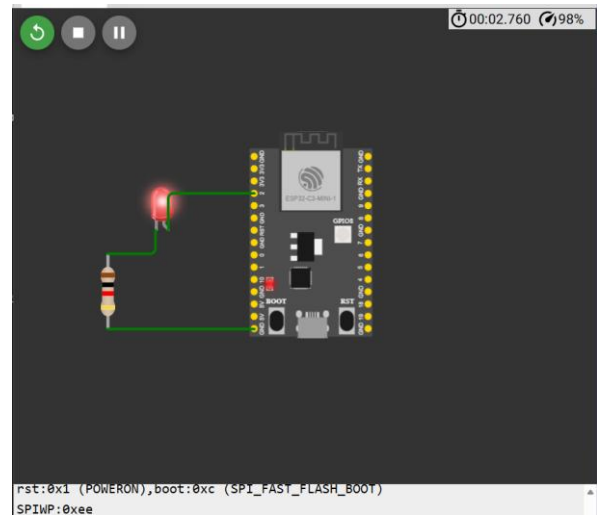
    # Ghi 1 vào các bit 2, 3, 4 để kéo ba chân lên mức HIGH
    li a1, GPIO_OUT_W1TS_REG
    li a2, 0x1C
    sw a2, 0(a1)

loop_forever:
    j loop_forever
```

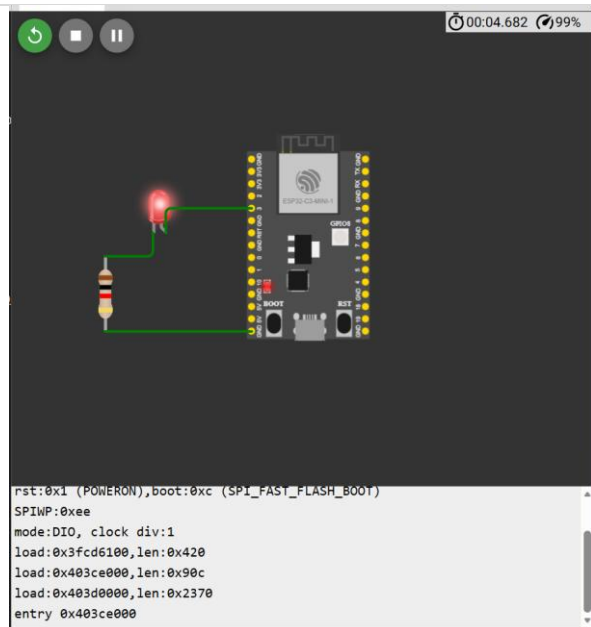
2. Kết quả mô phỏng



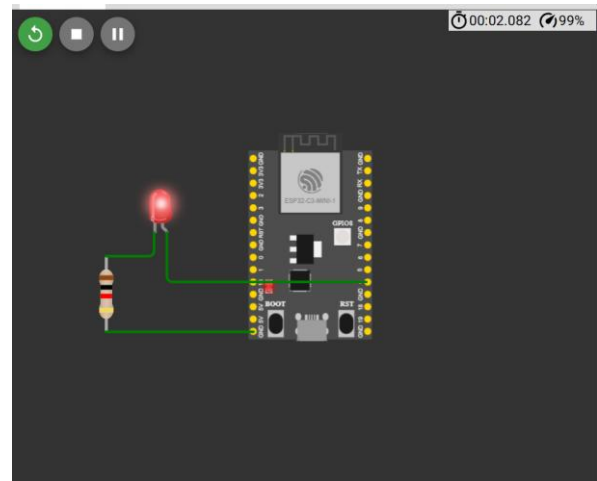
GPIO0 - đổi chiều trạng thái



GPIO2 phát sáng



GPIO3 phát sáng



GPIO4 phát sáng

3. Nhận xét và giải thích

- Giá trị 0x1C được dùng như một mặt nạ bit. Khi viết ở dạng nhị phân, giá trị này là 11100, vì vậy thao tác sẽ tác động đồng thời đến GPIO2, GPIO3 và GPIO4.
- GPIO_ENABLE_REG có nhiệm vụ xác định các chân nào sẽ được dùng làm ngõ ra. Sau khi ghi 0x1C, ba chân tương ứng được kích hoạt ở chế độ Output.

- GPIO4 cần cấu hình lại tại thanh ghi IO_MUX vì chân này có thể đang được gán cho chức năng ngoại vi khác. Việc ghi 0x1000 giúp chuyển chân về chức năng GPIO.
- Thanh ghi GPIO_OUT_W1TS_REG hoạt động theo cơ chế Write 1 To Set: bit nào được ghi bằng 1 thì chân GPIO tương ứng xuất mức cao, làm LED sáng.

Assignment 2 - Điều khiển ba LED nhấp nháy đồng thời

1. Mã nguồn đã biên tập chú thích

```
.global init

# Các thanh ghi phục vụ cấu hình và điều khiển GPIO
.equ GPIO_ENABLE_REG, 0x60004020
.equ GPIO_OUT_W1TS_REG, 0x60004008 # Set: ghi 1 để bật mức HIGH
.equ GPIO_OUT_W1TC_REG, 0x6000400C # Clear: ghi 1 để đưa về LOW
.equ IO_MUX_GPIO4_REG, 0x60004014

.text
init:
    # Cho phép GPIO2, GPIO3 và GPIO4 xuất tín hiệu
    li a1, GPIO_ENABLE_REG
    li a2, 0x1C
    sw a2, 0(a1)

    # Cấu hình riêng GPIO4 để bảo đảm chân này làm việc như GPIO thông thường
    li a1, IO_MUX_GPIO4_REG
    li a2, 0x1000
    sw a2, 0(a1)

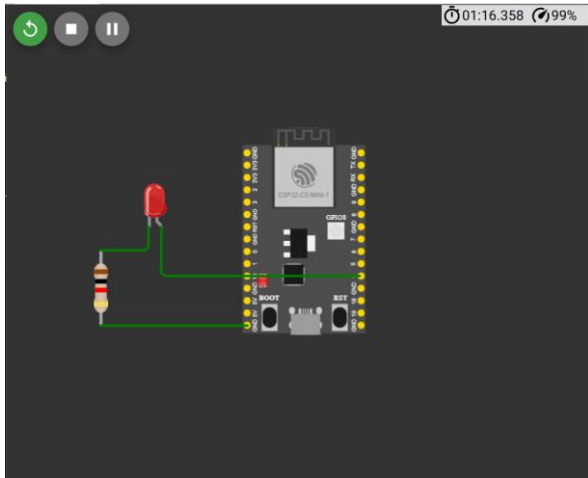
main_loop:
    # Bật đồng thời ba LED
    li a1, GPIO_OUT_W1TS_REG
    li a2, 0x1C
    sw a2, 0(a1)
    call delay_asm

    # Tắt đồng thời ba LED
    li a1, GPIO_OUT_W1TC_REG
    li a2, 0x1C
    sw a2, 0(a1)
    call delay_asm

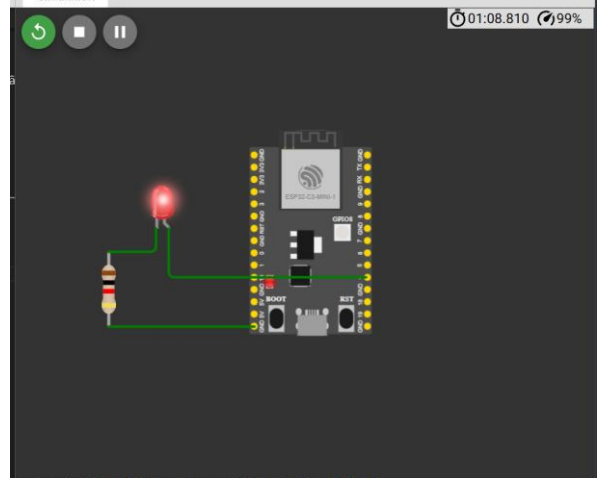
    j main_loop

# Tạo khoảng chờ bằng vòng lặp đếm trong hợp ngữ
delay_asm:
    li a3, 0
    li a4, 8000000 # Tăng giá trị này để LED nhấp chậm hơn, giảm để nhấp nhanh hơn
loop_delay:
    addi a3, a3, 1
    blt a3, a4, loop_delay
    ret
```

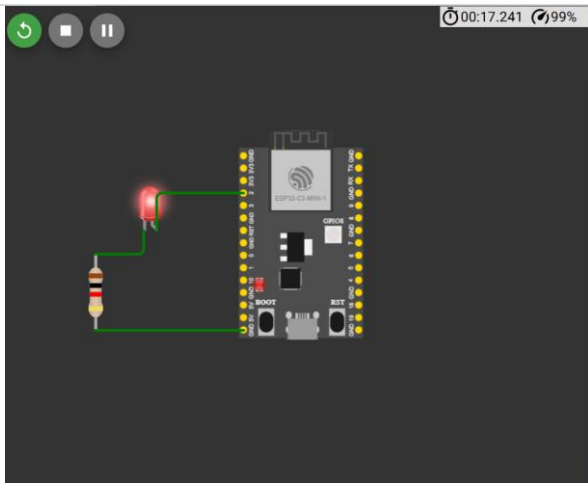
2. Kết quả mô phỏng



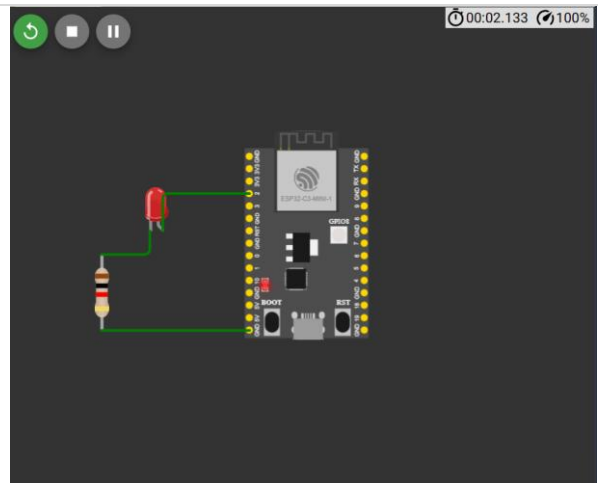
GPIO4 - trạng thái bật/tắt



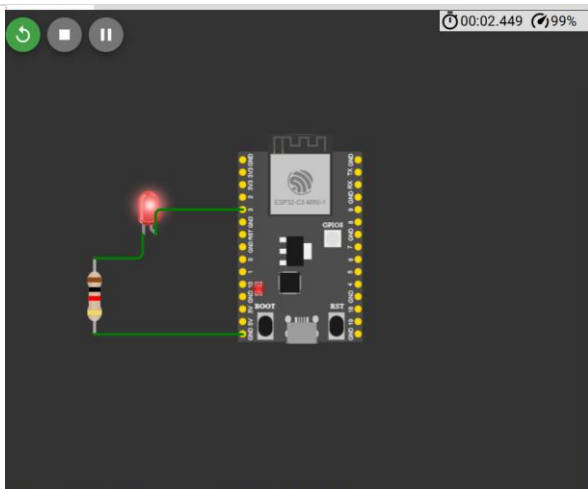
GPIO2 - trạng thái bật



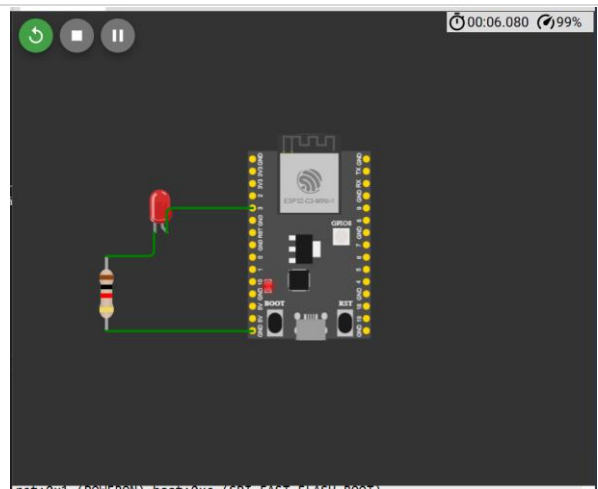
GPIO3 - trạng thái bật



GPIO2 - trạng thái tắt/bật



GPIO3 - trạng thái mô phỏng



GPIO4 - trạng thái mô phỏng

3. Phân tích hoạt động

- Toàn bộ ba LED được điều khiển chung bằng mặt nạ 0x1C, nhờ đó chương trình không cần xử lý từng chân riêng lẻ.
- Lệnh ghi vào W1TS đặt các chân lên mức HIGH, còn lệnh ghi vào W1TC đưa các chân về mức LOW.
- Hai thao tác bật và tắt được đặt trong main_loop nên quá trình lặp lại liên tục, tạo hiệu ứng nhấp nháy.
- Độ trễ được quyết định bởi số lần lặp trong delay_asm. Với giá trị 8.000.000, thời gian chờ dài hơn nên LED chuyển trạng thái chậm và dễ quan sát.

Assignment 3 - Hiển thị chữ số trên LED 7 đoạn

1. Mã nguồn đã biên tập chú thích

```
.global init

.equv GPIO_ENABLE_REG, 0x60004020
.equv GPIO_OUT_REG, 0x60004004
.equv IO_MUX_GPIO4_REG, 0x60009014
.equv IO_MUX_GPIO5_REG, 0x60009018
.equv IO_MUX_GPIO6_REG, 0x6000901C
.equv IO_MUX_GPIO7_REG, 0x60009020

.text
init:
    # Cho phép toàn bộ nhóm chân GPIO0 đến GPIO7 xuất tín hiệu
    li a1, GPIO_ENABLE_REG
    li a2, 0xFF
    sw a2, 0(a1)

    # Các chân 4, 5, 6, 7 cần được chuyển sang chế độ GPIO
    li a2, 0x1000
    li a1, IO_MUX_GPIO4_REG
    sw a2, 0(a1)
    li a1, IO_MUX_GPIO5_REG
    sw a2, 0(a1)
    li a1, IO_MUX_GPIO6_REG
    sw a2, 0(a1)
    li a1, IO_MUX_GPIO7_REG
    sw a2, 0(a1)

    # Xuất mã điều khiển để hiển thị số 0
    # Quy ước bit: F=bit6, G=bit5, E=bit4, D=bit3, C=bit2, B=bit1, A=bit0
    # Số 0 cần bật A, B, C, D, E, F và tắt G -> 0x5F
    li a1, GPIO_OUT_REG
    li a2, 0x5F
    sw a2, 0(a1)

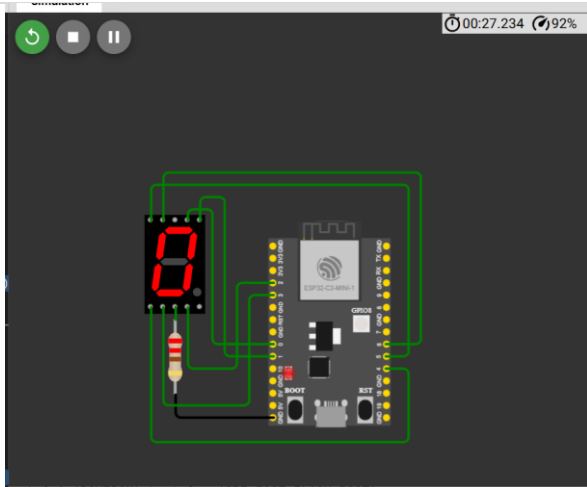
loop:
    j loop
```

2. Bảng mã điều khiển LED 7 đoạn

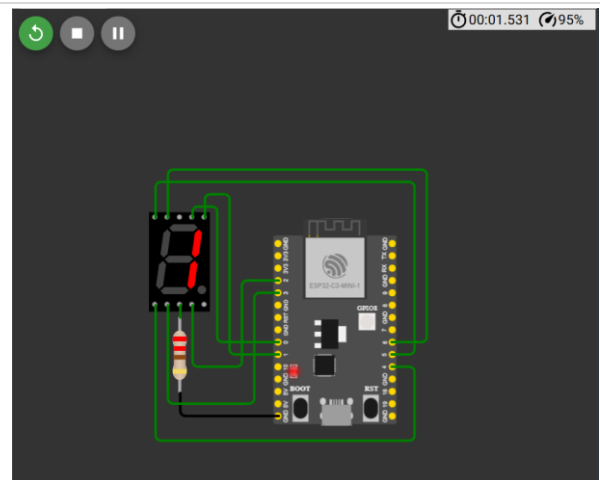
Chữ số	Các đoạn sáng	Mẫu bit theo thứ tự F-G-E-D-C-B-A	Mã Hex
0	A, B, C, D, E, F	1011111	0x5F
1	B, C	0000110	0x06
2	A, B, G, E, D	0111011	0x3B
3	A, B, G, C, D	0101111	0x2F
4	F, G, B, C	1100110	0x66
5	A, F, G, C, D	1101101	0x6D
6	A, F, E, D, C, G	1111101	0x7D
7	A, B, C	0000111	0x07
8	A, B, C, D, E, F, G	1111111	0x7F

9	A, B, C, D, F, G	1101111	0x6F
---	------------------	---------	------

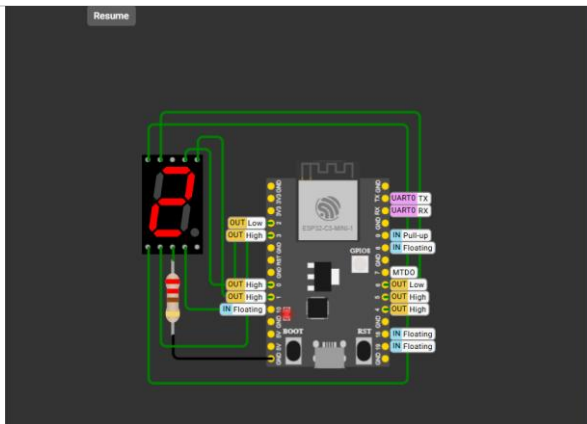
3. Kết quả mô phỏng



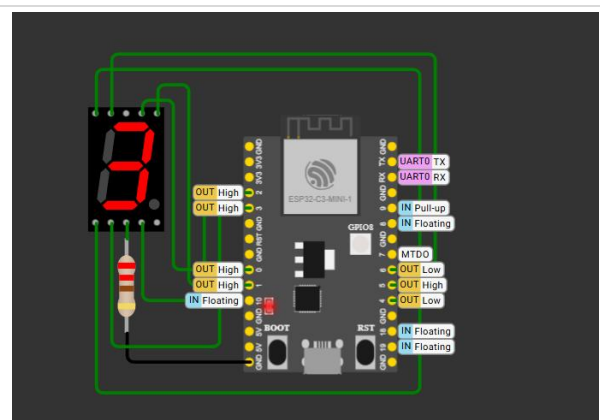
$A2 = 0xFF$ / kiểm tra toàn bộ đoạn



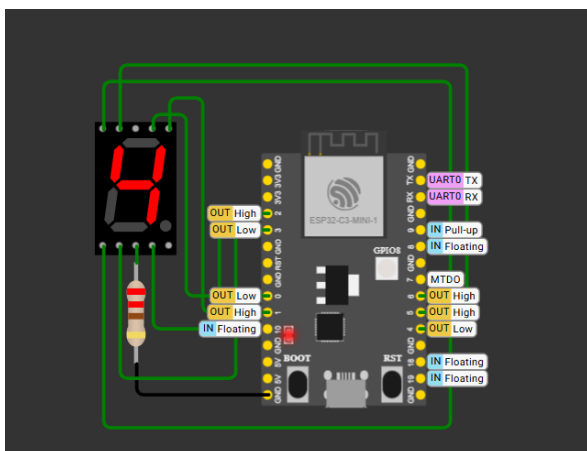
$A2 = 0x06$ / hiển thị số 1



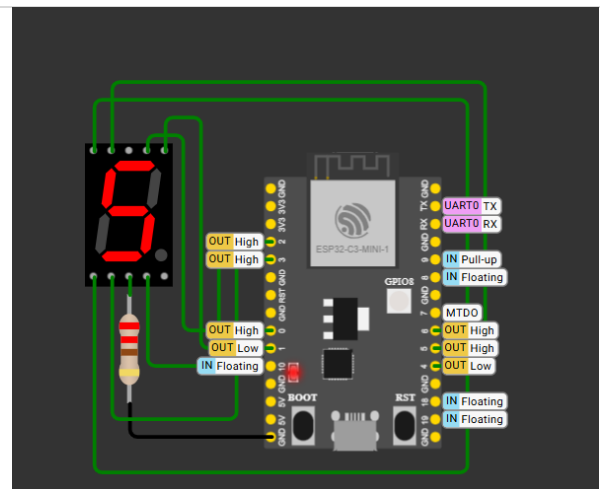
Hiển thị số 2



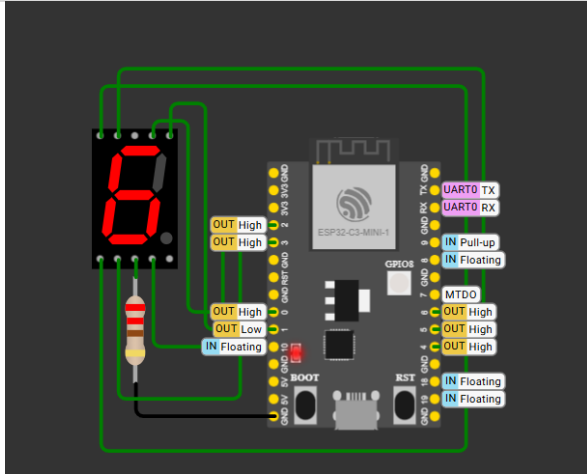
Hiển thị số 3



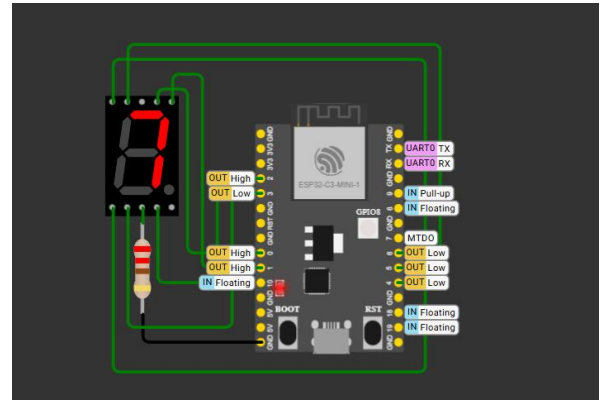
Hiển thị số 4



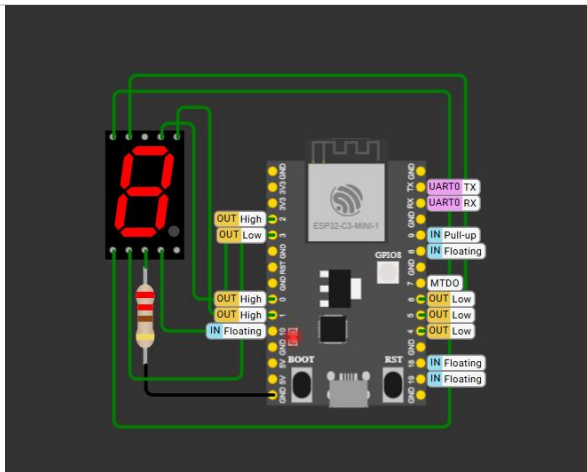
Hiển thị số 5



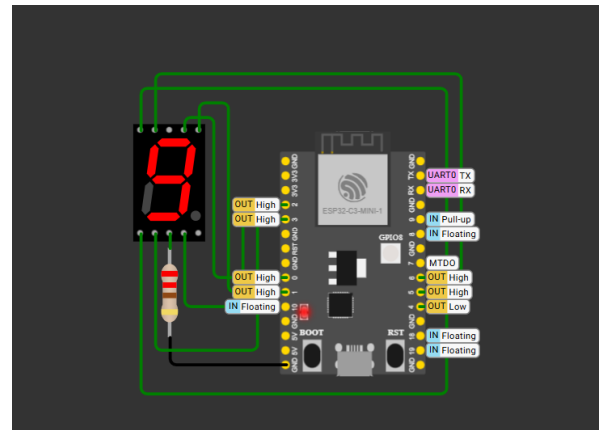
Hiển thị số 6



Hiển thị số 7



Hiển thị số 8



Hiển thị số 9

4. Giải thích

- LED 7 đoạn được điều khiển bằng cách gán mỗi đoạn A đến G cho một bit GPIO tương ứng.
- Khi nạp một mã Hex vào GPIO_OUT_REG, các bit 1 sẽ làm những đoạn tương ứng sáng, còn bit 0 giữ đoạn ở trạng thái tắt.
- Với cấu hình của bài, một chữ số bất kỳ được tạo ra bằng cách chọn đúng tổ hợp đoạn sáng theo bảng mã phía trên.
- Các chân GPIO4 đến GPIO7 được cấu hình lại để tránh xung đột với chức năng ngoại vi mặc định.

Assignment 4 - Đọc công tắc và điều khiển LED

1. Mã nguồn đã biên tập chú thích

```
.global init

# Địa chỉ thanh ghi sử dụng trong bài
.eqv GPIO_OUT_W1TS_REG, 0x60004008    # Đưa ngõ ra lên mức HIGH
.eqv GPIO_OUT_W1TC_REG, 0x6000400C    # Đưa ngõ ra về mức LOW
.eqv GPIO_ENABLE_REG, 0x60004020      # Chọn chân làm Output
.eqv GPIO_IN_REG, 0x6000403C          # Đọc dữ liệu từ các chân GPIO
.eqv IO_MUX_GPIO0_REG, 0x60009004     # Cấu hình chức năng đầu vào cho GPIO0

.text
init:
    # GPIO1 được dùng để điều khiển LED nên cần cấu hình là Output
    li a1, GPIO_ENABLE_REG
    li a2, 0x02                        # Bit 1 ứng với GPIO1
    sw a2, 0(a1)

    # GPIO0 dùng làm chân đọc công tắc
    # Bật FUN_IE tại bit 9 để cho phép nhận tín hiệu đầu vào
    li a1, IO_MUX_GPIO0_REG
    lw a2, 0(a1)
    li t0, 0x200
    or a2, a2, t0
    sw a2, 0(a1)

loop:
    # Đọc trạng thái hiện tại của các chân GPIO
    li a1, GPIO_IN_REG
    lw a2, 0(a1)

    # Tách riêng bit 0 để xác định trạng thái công tắc tại GPIO0
    andi a3, a2, 0x01

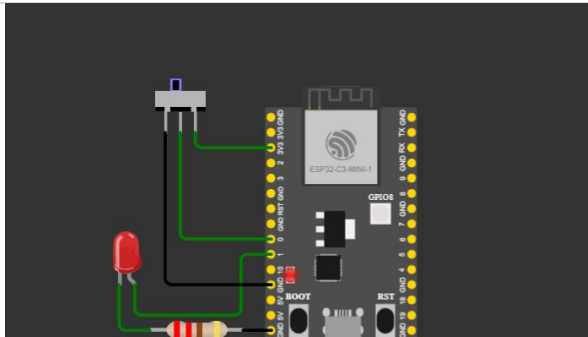
    # Nếu công tắc kéo GPIO0 xuống 0 thì tắt LED, ngược lại bật LED
    beq a3, zero, led_off

led_on:
    li a1, GPIO_OUT_W1TS_REG
    li a2, 0x02
    sw a2, 0(a1)
    j next

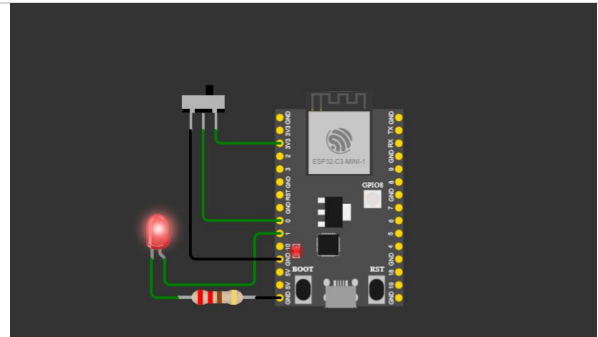
led_off:
    li a1, GPIO_OUT_W1TC_REG
    li a2, 0x02
    sw a2, 0(a1)

next:
    j loop
```

2. Kết quả mô phỏng



Công tắc ở trạng thái làm LED tắt



Công tắc ở trạng thái làm LED sáng

3. Mô tả hoạt động

- GPIO1 là chân xuất tín hiệu để điều khiển LED, còn GPIO0 được dùng để đọc trạng thái công tắc.
- Chương trình liên tục đọc GPIO_IN_REG rồi dùng phép AND với 0x01 để chỉ lấy riêng bit của GPIO0.
- Khi GPIO0 nhận mức 0, chương trình nhảy tới nhánh led_off và ghi vào W1TC để tắt LED.
- Khi GPIO0 ở mức 1, nhánh led_on được thực hiện và W1TS đặt GPIO1 lên mức HIGH để LED sáng.
- Vòng lặp chính giúp trạng thái LED thay đổi ngay khi công tắc được gạt.

Assignment 5 - Đếm số 0 đến 9 trên LED 7 đoạn

1. Mã nguồn đã biên tập chú thích

```
.global init

.equv GPIO_ENABLE_REG, 0x60004020
.equv GPIO_OUT_REG, 0x60004004
.equv IO_MUX_GPIO4_REG, 0x60009014
.equv IO_MUX_GPIO5_REG, 0x60009018
.equv IO_MUX_GPIO6_REG, 0x6000901C

.text
init:
    # Cho phép nhóm GPIO0 đến GPIO7 làm ngõ ra để điều khiển LED 7 đoạn
    li a1, GPIO_ENABLE_REG
    li a2, 0xFF
    sw a2, 0(a1)

    # Cấu hình các chân GPIO4, GPIO5 và GPIO6 về chế độ GPIO
    li a2, 0x1000

    li a1, IO_MUX_GPIO4_REG
    sw a2, 0(a1)

    li a1, IO_MUX_GPIO5_REG
    sw a2, 0(a1)

    li a1, IO_MUX_GPIO6_REG
    sw a2, 0(a1)

main:
    # Lần lượt xuất các mã hiển thị từ 0 đến 9
    li a1, GPIO_OUT_REG

    li a2, 0xC0    # 0
    sw a2, 0(a1)
    call delay

    li a2, 0xF9    # 1
    sw a2, 0(a1)
    call delay

    li a2, 0xA4    # 2
    sw a2, 0(a1)
    call delay

    li a2, 0xB0    # 3
    sw a2, 0(a1)
    call delay

    li a2, 0x99    # 4
    sw a2, 0(a1)
    call delay
```

```

li a2, 0x92      # 5
sw a2, 0(a1)
call delay

li a2, 0x82      # 6
sw a2, 0(a1)
call delay

li a2, 0xF8      # 7
sw a2, 0(a1)
call delay

li a2, 0x80      # 8
sw a2, 0(a1)
call delay

li a2, 0x90      # 9
sw a2, 0(a1)
call delay

j main

```

Hàm trễ giúp mỗi số được giữ trên màn hình trong một khoảng thời gian

delay:

```

li t0, 0
li t1, 3000000

```

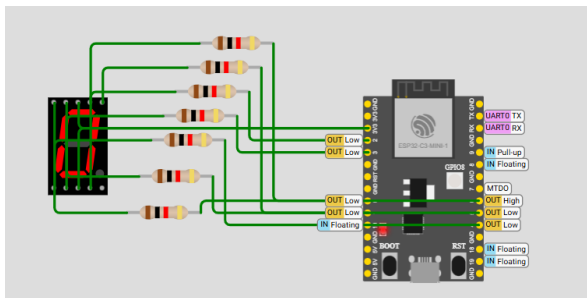
loop:

```

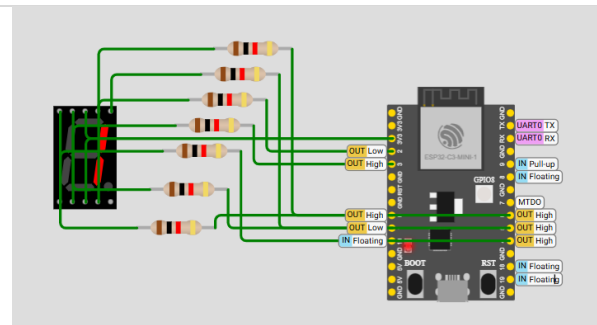
addi t0, t0, 1
blt t0, t1, loop
ret

```

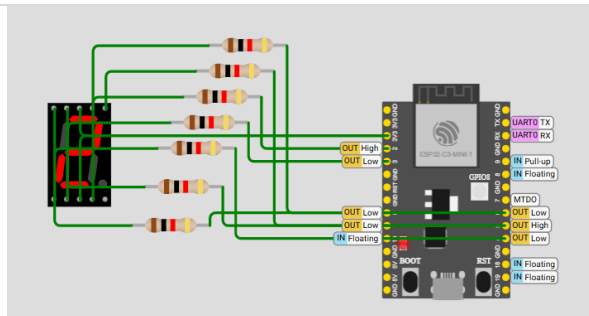
2. Kết quả mô phỏng



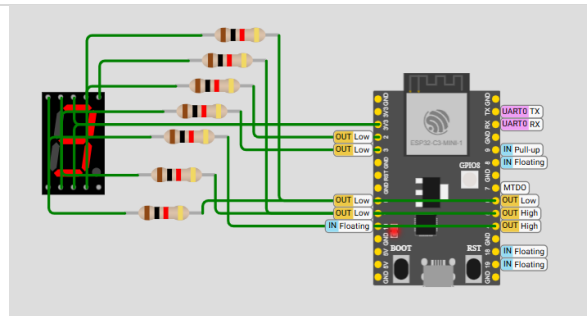
Hiển thị số 0



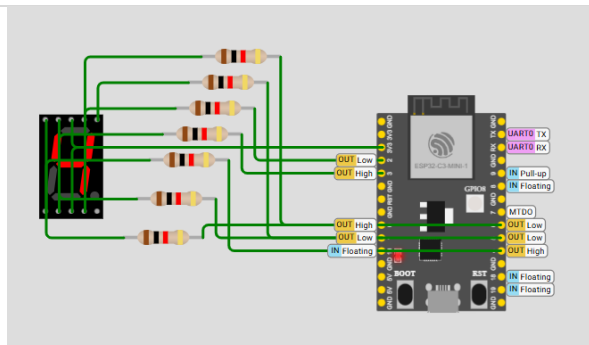
Hiển thị số 1



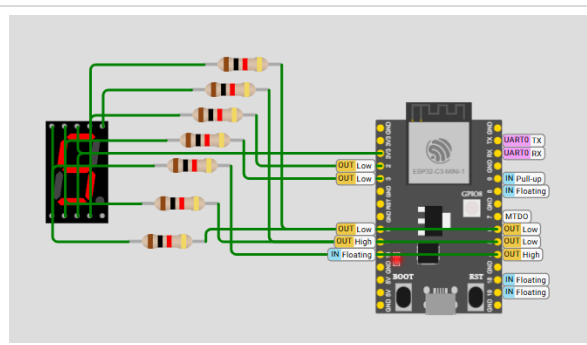
Hiển thị số 2



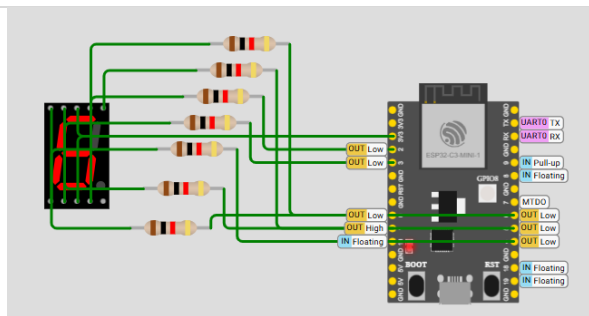
Hiển thị số 3



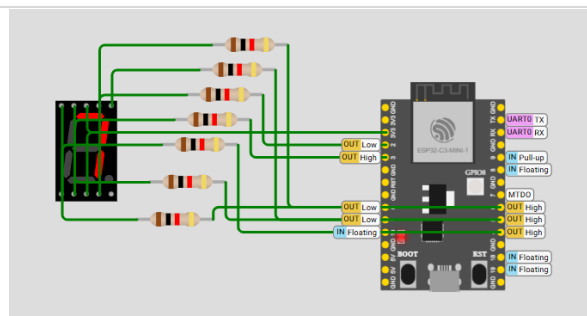
Hiển thị số 4



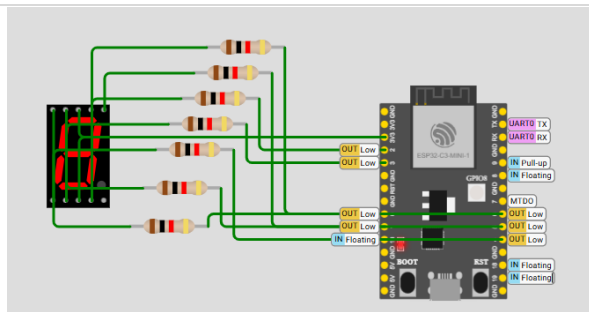
Hiển thị số 5



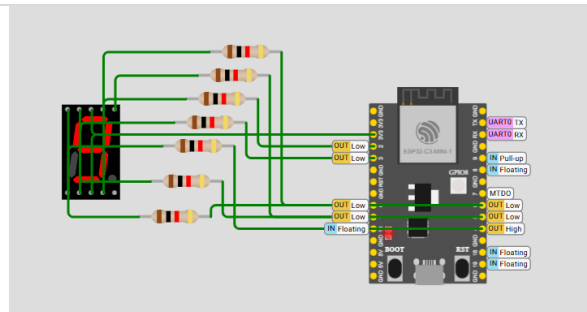
Hiển thị số 6



Hiển thị số 7



Hiển thị số 8



Hiển thị số 9

3. Giải thích quy trình

1. Đầu tiên, chương trình cấu hình GPIO0 đến GPIO6 làm chân Output để nối với các đoạn a đến g của LED.

2. Mỗi chữ số có một mã điều khiển riêng. Mã này được ghi trực tiếp vào GPIO_OUT_REG để bật hoặc tắt các đoạn cần thiết.
3. Sau khi một số được hiển thị, chương trình gọi delay để giữ hình ảnh đủ lâu cho người quan sát.
4. Khi hết thời gian chờ, mã của số kế tiếp được xuất ra.
5. Sau khi hiển thị đến số 9, lệnh j main đưa chương trình quay về đầu dãy và tiếp tục đếm lại từ 0.
6. Nhờ vòng lặp vô hạn, LED 7 đoạn tạo thành bộ đếm tuần hoàn từ 0 đến 9.

Kết luận

Qua năm phần thực hành, báo cáo đã minh họa các thao tác cơ bản khi lập trình hợp ngữ cho ESP32-C3: cấu hình GPIO, ghi mức logic ra chân, đọc trạng thái đầu vào, tạo trễ bằng vòng lặp và điều khiển LED 7 đoạn. Bố cục mới tách rõ mã nguồn, ảnh kết quả và phần giải thích để người đọc dễ kiểm tra từng yêu cầu của bài.